Image Fragment Reconstruction via Adjacency Prediction

May 4, 2026

Contents

1 Problem Statement

Each batch contains $N = 10$ source images of resolution $64 \times 64 \times 3$. Every image is cut into a regular 4×4 grid of non-overlapping 16×16 fragments, and the resulting $M = 160$ fragments from all 10 images are pooled into a single unordered collection. The model must partition the collection into 10 groups such that all fragments in a group come from the same source image.

Two constraints make the task non-trivial. First, the original grid coordinates of each fragment are withheld at inference, so absolute position cannot be used as a feature. Second, the source-image label of each fragment, which we denote $s(k) \in \{1, \ldots, 10\}$, is withheld during training, so the model has to learn from structural properties of the fragments rather than from direct supervision. Two facts about the partition are known and may be exploited: there are exactly 10 clusters, and each contains exactly 16 fragments.

Crucially, although the source labels are hidden, *spatial adjacency* within a source image is fully determined by the grid: two fragments at grid positions (r, c) and (r', c') within the same source image are adjacent iff $|r - r'| + |c - c'| = 1$. This adjacency relation is therefore a free-of-charge supervisory signal — and the one our pretext task exploits (Section 2).

2 Related Work

The problem sits at the intersection of self-supervised representation learning and clustering. We review three families of prior work that informed the approach.

Pretext-task self-supervision. The dominant paradigm in self-supervised vision until the rise of contrastive methods was to define a hand-crafted pretext task whose labels are derivable from the data itself. Noroozi and Favaro (2016) solve a jigsaw permutation: given a 3×3 grid of fragments shuffled by one of 100 fixed permutations, predict which permutation was used. The encoder learns features useful for downstream classification. Our adjacency prediction is a much simpler, binary variant of this idea — rather than predicting a discrete permutation index over an exponentially large space, we predict a binary label per pair. This trades off representational richness for sample efficiency.

Contrastive learning. SimCLR (Chen et al. 2020) and MoCo (He et al. 2020) learn representations by pulling together augmented views of the same image and pushing apart views of different images. In the present setting these methods are awkward: there is no natural positive pair construction, since the natural "view" of a fragment is the fragment itself. One could treat all fragments from one source image as positives of each other, but this requires the source label that we are explicitly forbidden to use during training. Adjacency labels, in contrast, are derivable from the grid structure alone.

Clustering-based self-supervision. DeepCluster (Caron, Bojanowski, et al. 2018) alternates between clustering current embeddings and using cluster assignments as pseudo-labels. SwAV (Caron, Misra, et al. 2020) extends this with online clustering and equipartitioned cluster assignments via the Sinkhorn algorithm. Both are conceptually well-suited to this task, but they introduce significant complexity (alternating training, prototype maintenance) for what is at heart a small-scale clustering problem with $N = 10$ images. We treat this family as a strong candidate for follow-up work but defer it.

3 Core Idea

The central observation is that two fragments cut from the same image and placed next to each other in the grid share a common boundary — colours bleed over, textures continue, and objects span tiles. This visual continuity is a strong and reliable signal that does not require any human annotation to exploit: since the experimenter creates the grid, the ground-truth adjacency of every fragment pair is known at all times.

The proposed approach uses adjacency prediction as a **pretext task** (Noroozi and Favaro 2016). The model is trained to solve a surrogate problem — predicting whether two fragments are spatially adjacent — and in doing so is forced to learn visual representations that capture source identity as a side effect. At inference time the pretext objective is discarded and the learned representations are used for clustering.

This is self-supervised throughout: no labels beyond those derivable from the grid structure are ever used.

4 Architecture

The model is a Siamese network: a shared encoder is applied to each of two input fragments, and a single comparison head maps the two embeddings to a confidence score $p_{ij} \in [0, 1]$ (Figure 1). The same score is used for both loss terms during training and as the clustering similarity at inference.

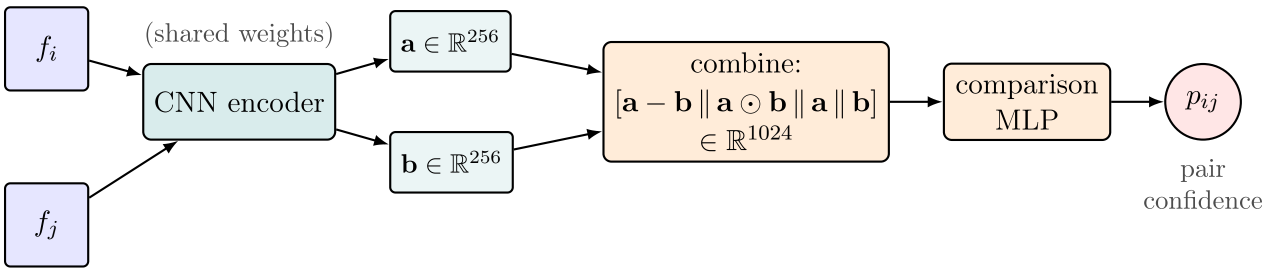


Figure 1: Schematic of the model. The CNN encoder is applied with shared weights to each of two input fragments; its 256-dimensional embeddings are combined into a single 1024-dimensional vector via concatenation of difference, element-wise product, and the two embeddings themselves; this vector is fed to a single comparison head that produces one confidence score $p_{ij} \in [0, 1]$ per pair.

The components are:

Shared CNN encoder. A single convolutional neural network maps each 16×16×3 fragment to a 256-dimensional embedding vector. The architecture is three convolutional blocks (Conv–BatchNorm–ReLU), with max-pooling after blocks two and three, reducing the spatial dimensions from 16×16 to 4×4. The flattened $4 \times 4 \times 128$ feature map is projected to a 256-dimensional embedding via a linear layer followed by ReLU and dropout ($p = 0.3$).

The encoder is **shared** — the same weights are used for both fragments in a pair. This is the defining property of a Siamese network (Bromley et al. 1994): weight sharing ensures that the comparison is symmetric and that the encoder learns a universal fragment representation rather than one tuned to a specific input slot.

Comparison head (MLP). Let $\mathbf{a} = \text{CNN}(f_i)$ and $\mathbf{b} = \text{CNN}(f_j)$ denote the two embeddings produced by the shared encoder for an input pair. The comparison head takes as input the 1024-dimensional vector

$$\mathbf{z}_{ij} = \big[\mathbf{a} - \mathbf{b} \parallel \mathbf{a} \odot \mathbf{b} \parallel \mathbf{a} \parallel \mathbf{b}\big] \in \mathbb{R}^{1024},$$

formed by concatenating ($\parallel$) four sub-vectors of size 256: the element-wise difference $\mathbf{a} - \mathbf{b}$, capturing direction and magnitude of dissimilarity; the element-wise (Hadamard) product $\mathbf{a} \odot \mathbf{b}$, capturing feature co-activation (large only when both fragments score high on the same feature); and the two raw embeddings, preserving each fragment's individual information. This four-term combination is borrowed from the InferSent sentence-pair classification model of Conneau et al. (2017), where it was shown to outperform plain concatenation on natural-language inference. We adopt it here without ablation in our setting; whether the explicit difference and product terms help on image fragments, or whether a simpler $[\mathbf{a}, \mathbf{b}]$ concatenation with a larger hidden layer would do as well, is left as future work. The MLP maps $\mathbf{z}_{ij}$ through that hidden layer (ReLU) to a single scalar in $[0, 1]$:

$$p_{ij} = \sigma\big(W_2 \,\text{ReLU}(W_1 \,\mathbf{z}_{ij} + b_1) + b_2\big).$$

5 Training

The training set consists of 1,281,167 images from ImageNet-64. Each training step samples 10 images independently and uniformly at random, so the data generator is effectively infinite and no epoch structure is imposed.

5.1 Constructing Training Labels

From a sample of 10 images cut into a 4×4 grid, each fragment has at most 4 adjacent neighbours (fewer at corners and edges). For a sample of 160 fragments the total number of pairs is:

$$\binom{160}{2} = 12{,}720$$

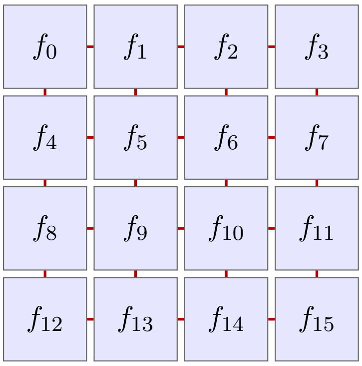
Each pair receives a binary label: $y_{ij} = 1$ if the two fragments are adjacent within the same image, $y_{ij} = 0$ otherwise. This label is derived entirely from the known grid positions and requires no human annotation (Figure 2).

5.2 Loss Function

Binary cross-entropy is the natural loss for a binary prediction task:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{(i,j)} \Big[y_{ij} \log p_{ij} + (1 - y_{ij}) \log(1 - p_{ij}) \Big]$$

Positive (adjacent) pairs are far rarer than negative pairs in this configuration. With 10 source images on a 4×4 grid, each image contributes $4 \cdot 3 + 3 \cdot 4 = 24$ adjacent pairs, giving $n_+ = 240$ positives in total. The remaining pairs are negatives: $n_- = \binom{160}{2} - 240 = 12{,}480$. The ratio $n_-/n_+ = 52$ is therefore a constant determined by the grid geometry, not a per-batch quantity. Vanilla BCE on this data is dominated by the negative class: the



$y_{ij} = 1$**:** f_i and f_j adjacent (red edge)

$y_{ij} = 0$**:** all other pairs

16 fragments per image

$4 \cdot 3 + 3 \cdot 4 = 24$ adjacent pairs

$\binom{16}{2} - 24 = 96$ non-adjacent same-image pairs

Figure 2: Adjacency labelling within a single source image. Each of the 16 fragments is connected by a red edge to each of its grid neighbours. The 24 highlighted edges are the positive pairs ($y_{ij} = 1$); all other pairs within the image (96 of them) and across images are negative ($y_{ij} = 0$).

gradient is overwhelmingly driven by the loss on non-adjacent pairs, and the model has little incentive to identify the few adjacent ones. The standard remedy is to upweight the rare class. Since only the ratio of the two class weights matters, we parameterise the loss with a single *tilt* parameter $\beta \geq 1$:

$$\mathcal{L}_{\text{WBCE}}(\beta) = -\frac{1}{N} \sum_{(i,j)} \Big[\beta \cdot \tfrac{n_-}{n_+}\, y_{ij} \log p_{ij} + (1 - y_{ij}) \log(1 - p_{ij}) \Big]. \tag{1}$$

The textbook value is $\beta = 1$, the class-balanced loss: it equalises the total gradient contribution of the two classes, treating a single positive as worth 52 negatives. The plain (unweighted) BCE corresponds to $\beta = 1/52$. Empirically (Section 9.3) β turns out to be a second-order knob in this range and we adopt $\beta = 1/52$ throughout.

Multi-task loss with same-image prediction. There is a more fundamental mismatch in the loss above: adjacency is a *local* signal (only 24 pairs per image), while the downstream task is *global* clustering by source image. Two fragments from opposite corners of the same source image are penalised as a negative example by adjacency BCE, even though the clustering wants them in the same group. This mismatch limits how well any model trained purely on adjacency can do.

The natural fix is to add a second loss term on a denser, more task-aligned signal: same-image membership. The label

$$y_{ij}^{\text{same}} = \begin{cases} 1 & \text{if } s(i) = s(j) \\ 0 & \text{otherwise} \end{cases}$$

is also free, derivable from the labels passed to the data generator, and has a $\sim 10\%$ positive rate (versus 1.9% for adjacency). Both terms act on the *same* per-pair score p_{ij} from the single comparison head:

$$\mathcal{L}_{\text{multi}} = \lambda_{\text{adj}}\, \mathcal{L}_{\text{WBCE}}^{\text{adj}}(\beta_{\text{adj}}) + \lambda_{\text{same}}\, \mathcal{L}_{\text{BCE}}^{\text{same}}. \tag{2}$$

The adjacency term uses $\beta_{\text{adj}} = 1/52$ (plain BCE; the class imbalance problem motivating β is specific to adjacency with its $\sim$1.9% positive rate). The same-image term uses unweighted BCE: at $\sim$10% positives the imbalance is mild and no upweighting is needed.

At inference, p_{ij} is used directly as the similarity matrix for spectral clustering. The choice of weights is ablated in Section 9.4.

The two signals are complementary in the sense documented by the same-image-only results in Section 9.5: adjacency provides a fine-grained relational signal that prevents the encoder from collapsing within an image, while same-image provides the dense task-aligned signal that the clustering step actually consumes.

5.3 Computational Cost

The CNN encoder runs once per fragment (160 forward passes per sample). All pairwise embeddings are then combined using tensor operations, and the lightweight MLP processes all 12,720 pairs. Since the expensive step (the CNN) runs only 160 times, the computational overhead is manageable.

5.4 Validation: multi-batch averaging

Each validation step samples a fresh batch of 10 source images from the test generator. Individual batches vary substantially in difficulty (some draws of 10 ImageNet classes are visually well-separated, others contain near-duplicates), so a single-batch ARI estimate has high variance. We average each downstream metric (ARI, NMI; formally defined in Section 7) and each pretext-task metric (AUROC, AUPRC) over $n_{\text{eval}} = 20$ fresh validation batches per evaluation step. This stabilises the training curves, and — more importantly — the early-stopping decision and the checkpoint-selection criterion, which would otherwise be biased toward lucky batches.

6 From Pretext Task to Clustering

After training, the model produces a value $p_{ij} \in [0, 1]$ for every pair of fragments. We do not interpret this as a calibrated probability — the model is trained on a binary cross-entropy objective but its outputs are not guaranteed to be calibrated to true frequencies. We use p_{ij} instead as a confidence score: the higher the value, the more confident the model is that fragments i and j are adjacent. These scores define a **fully connected weighted graph** over the 160 fragments, where edge weight $w_{ij} = p_{ij}$ reflects the model's confidence that the two fragments are adjacent — and by extension, that they originate from the same image (Figure 3).

Balanced spectral clustering. **Spectral clustering** is the natural choice for a weighted graph. Given the 160×160 similarity matrix W with $w_{ij} = p_{ij}$, define the diagonal degree matrix D with $D_{ii} = \sum_j w_{ij}$ and the symmetric normalised graph Laplacian

$$L_{\text{sym}} = I - D^{-1/2} W D^{-1/2}.$$

The spectral embedding stacks the k leading eigenvectors of L_{sym} into a $160 \times k$ matrix; in the eigenspace, fragments belonging to densely-connected sub-graphs lie close together. The standard pipeline then runs k-means on this embedding to obtain hard cluster assignments.

Plain k-means ignores the fact that each true cluster contains exactly $G^2 = 16$ fragments and can produce groups of very unequal size. We enforce balance by replacing the

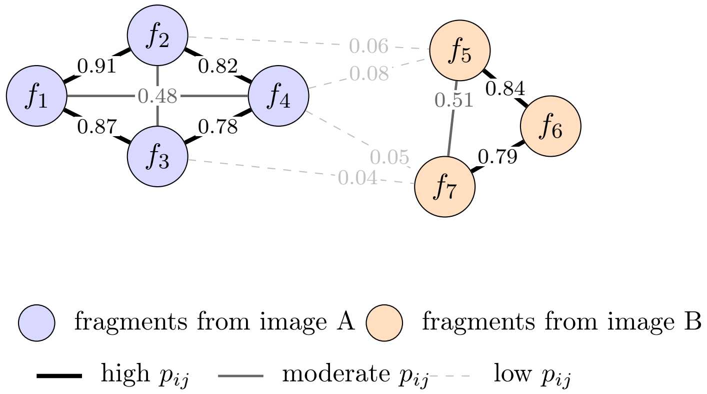


Figure 3: Schematic of the predicted similarity graph. Each node is a fragment; edge weight p_{ij} is the model's confidence that the two fragments are adjacent. Edges within a true source image (here, two illustrative source images shown in blue and orange) tend to receive high weight; cross-image edges receive low weight. Spectral clustering recovers the source-image partition by finding the cut that separates the densely-connected subgraphs.

k-means step with a balanced assignment: build a $160 \times (k \cdot G^2)$ cost matrix in which each cluster appears G^2 times, and solve the resulting linear assignment problem with the Hungarian algorithm[1]. Cluster centroids are then re-estimated and the assignment iterated until convergence (typically a handful of iterations). The result is a partition of the 160 fragments into 10 clusters of size 16, by construction.

7 Evaluation Metric for the Clustering Task

The pretext task (adjacency prediction) and the downstream task (clustering of fragments by source image) are evaluated with different metrics. Adjacency prediction is evaluated with the threshold-free metrics AUROC and AUPRC (described below in the multi-batch averaging paragraph); the downstream clustering is evaluated with the **Adjusted Rand Index (ARI)**. We focus on the clustering metric here since it is the headline number for this work.

Definition (Adjusted Rand Index). Let $\mathcal{C} = \{C_1, \ldots, C_N\}$ be the true partition of the fragments into $N = 10$ source-image groups, and $\hat{\mathcal{C}} = \{\hat{C}_1, \ldots, \hat{C}_N\}$ the partition predicted by the model. For each pair of fragments, look at whether the two are in the same true group[2] and whether they are in the same predicted cluster:

- a = number of pairs in the same true group *and* the same predicted cluster
- b = number of pairs in different true groups *and* different predicted clusters
- c = number of pairs in the same true group but different predicted clusters

[1]Implemented using `sklearn.manifold.SpectralEmbedding` for the eigendecomposition and `scipy.optimize.linear_sum_assignment` for the assignment.

[2]For any pair of fragments (i, j), the two are in the *same true group* if $s(i) = s(j)$ (both came from the same source image), and in *different true groups* if $s(i) \neq s(j)$ (one came from image A, the other from image B).

- d = number of pairs in different true groups but the same predicted cluster

The (unadjusted) Rand Index is the agreement rate $\mathrm{RI} = (a+b)/(a+b+c+d)$. The Adjusted Rand Index normalises this against the expected RI under random labellings of fixed cluster sizes:

$$\mathrm{ARI} = \frac{\mathrm{RI} - \mathbb{E}[\mathrm{RI}]}{\max(\mathrm{RI}) - \mathbb{E}[\mathrm{RI}]}.$$

A random clustering scores ARI = 0, a perfect one scores ARI = 1, and a clustering worse than random can score below zero.

Definition (purity). *Purity* is the simpler clustering metric we report alongside ARI. It is the fraction of fragments that share their predicted cluster with the true plurality class of that cluster:

$$\mathrm{purity}(\hat{\mathcal{C}}, \mathcal{C}) = \frac{1}{M} \sum_{n=1}^{N} \max_{m} |\hat{C}_n \cap C_m|.$$

Purity ranges from 0 to 1. With our balanced spectral clustering enforcing exactly $G^2 = 16$ fragments per cluster, purity has no inflation toward 1 from over-fragmentation; it directly reads as "fraction of fragments correctly absorbed by their cluster's plurality source".

Why ARI rather than purity. ARI is preferred as the primary metric because purity only looks at the dominant true class in each predicted cluster and ignores how the remaining errors are distributed.

> **Example.** Suppose two predicted clusters each have 60% fragments from image A. The first additionally has 40% from image B; the second has 20% from B and 20% from C. Both clusters score the same purity (0.60), even though the second is visibly more disordered. ARI distinguishes the two cases because it counts pair-level agreements rather than looking only at the plurality label.

NMI (Normalised Mutual Information) is reported alongside ARI as a complementary metric. It measures how much information the predicted clusters share with the true groupings, normalised to $[0, 1]$.

8 Experimental Setup

Data. ImageNet-64 (Chrabaszcz, Loshchilov, and Hutter 2017): a downsampled version of the ImageNet-1k classification dataset in which every image has been resized to 64×64 pixels. The dataset is split into a training partition and a test partition; we draw the model's training images from the former and the validation images from the latter. Each training step samples 10 random images on the fly and produces a fresh batch of 160 fragments; the data generator is therefore effectively infinite. The augmentation pipeline (rotation up to $\pm 20°$, horizontal/vertical shift up to 10%, channel shift 0.2) is applied to the full image *before* fragmentation and is provided by the dataset's data generator (`data.py`); it is not modified.

Hardware. All training and evaluation reported in this document was performed on a MacBook Air (Apple M-series CPU). PyTorch is configured to fall back to CPU; no GPU is required. A full training run completes in approximately 20–25 minutes on this hardware.

Reproducibility.

- Each run writes its full configuration, training log, and best checkpoint into `runs/{run_name}/`, and appends a one-line summary to `results.jsonl`. The configuration is therefore self-contained and re-runnable.
- Random seeds are fixed for the clustering routines[3]. The PyTorch and NumPy seeds are controlled via the `seed` key in `config.yaml` (calling `torch.manual_seed` and `np.random.seed` at startup). When `seed` is omitted the run is unseeded, giving the natural run-to-run variance. All ANOVA replicates in Section 9.5 use explicit seeds $\{0, 1, 2, 3, 4\}$.
- Hyperparameters and run metadata are versioned in `config.yaml`.

Hyperparameters. The complete list of hyperparameters is maintained in `config.yaml` and may still change in the course of further ablations. The current values are documented there; this section will be replaced with a final hyperparameter table once the study is complete.

9 Results

Unless stated otherwise, all results in this section are reported on **validation data**: batches drawn from the held-out test split of ImageNet-64 that the model never trains on, but which are used for early stopping and checkpoint selection. A final unbiased evaluation on independent test batches is reported in Section 9.5.

9.1 Headline number and run-to-run noise

The best configuration we found uses a multi-task loss with equal weights on the adjacency and same-image terms, plain BCE on both, balanced spectral clustering, and validation ARI averaged over 20 fresh batches. We trained the same configuration five times with random seeds $\{0, 1, 2, 3, 4\}$ to estimate the run-to-run variance:

seed	0	1	2	3	4	**mean**	**std**
best ARI	0.575	0.545	0.571	0.512	0.502	0.541	**0.033**
final AUROC	0.960	0.958	0.954	0.952	0.947	0.954	0.005
final AUPRC	0.339	0.318	0.319	0.297	0.297	0.314	0.018

The best single seed achieves ARI = 0.575, the mean across 5 seeds is 0.541, and we use $\sigma_{\text{seed}} = 0.033$ as our noise floor. Differences smaller than ~ 0.066 (2σ) between configurations should be treated as inconclusive. AUROC is much more stable than ARI: σ on AUROC is only 0.005, an order of magnitude tighter. The model's underlying ranking quality is robust to seed choice; what varies is how cleanly that ranking translates into a discrete partition — the spectral clustering step is amplifying small representation differences into noticeable ARI differences.

[3] `random_state = 42` in `sklearn.cluster.SpectralClustering`, `sklearn.cluster.KMeans`, and `sklearn.manifold.SpectralEmbedding`.

9.2 Hyperparameter optimisation

Table 1 summarises the five conditions compared. Each condition is run with five fixed random seeds.

Model. Let y_{ij} denote the best-checkpoint ARI for condition $i \in \{a, b, c, d, e\}$ and seed $j \in \{0, \ldots, 4\}$. The one-way ANOVA model is

$$y_{ij} = \mu + \tau_i + \varepsilon_{ij}, \qquad \varepsilon_{ij} \sim \mathcal{N}(0,\, \sigma_i^2), \tag{3}$$

where μ is the grand mean, τ_i is the effect of condition i (with $\sum_i \tau_i = 0$), and ε_{ij} is the residual. The error variances σ_i^2 are allowed to differ across conditions (Welch one-way ANOVA[4]). The omnibus null hypothesis $H_0\colon \tau_a = \tau_b = \tau_c = \tau_d = \tau_e = 0$ is tested at $\alpha = 0.10$. Because no condition plays the role of a natural baseline, the p-value from the omnibus test is of limited interpretive value regardless of outcome: a non-significant result directly suggests the hyperparameter choice is not a strong lever, while a significant result requires follow-up to identify which conditions differ. The primary scientific output is therefore the effect size and confidence interval on each planned contrast.

Planned contrasts. Two contrasts are specified a priori and tested if the omnibus test is significant:

1. **Same-image loss:** $H_0\colon \tau_c = \tau_a$ — does adding the same-image loss term at $\lambda_{\text{same}} = 1$ improve clustering over adj-only?
2. **Loss tilt:** $H_0\colon \tau_e = \tau_a$ — does the textbook class-balanced weighting ($\beta = 1$) change performance relative to plain BCE ($\beta = 1/52$)?

Detectable effect sizes. With $n = 5$ seeds per condition, the standard error of the mean is $\hat{\sigma}/\sqrt{5}$, where $\hat{\sigma}$ is the pooled within-condition standard deviation estimated from the experiment. The minimum detectable difference in ARI at 80% power is approximately $1.43\,\hat{\sigma}$. The pooled within-condition standard deviation from the completed experiment is $\hat{\sigma} = 0.015$, giving $\Delta_{\min} \approx 0.022$. The observed inter-condition ARI differences among conditions (a)–(d) are 0.01–0.03, right at the detection boundary; condition (e) deviates by $\sim$0.08, well above it.

Results. The omnibus Welch one-way ANOVA is highly significant ($F(4,\, 9.75) = 22.6$, $p = 6.3 \times 10^{-5}$, $\alpha = 0.10$), so the planned contrasts are evaluated.

Contrast 1 (same-image loss, (c) vs (a)): $\Delta = +0.020$, 95% CI $[-0.001,\ 0.040]$, $p = 0.055$, $d = 1.49$ — significant at $\alpha = 0.10$. Adding the same-image loss yields a modest positive effect, but the confidence interval barely excludes zero, and the effect size is small in absolute ARI terms.

Contrast 2 (loss tilt, (e) vs (a)): $\Delta = -0.077$, 95% CI $[-0.103,\ -0.051]$, $p = 0.0001$, $d = -4.35$ — highly significant. Class-balanced weighting ($\beta = 1$) substantially degrades clustering performance relative to plain BCE ($\beta = 1/52$).

[4] A two-way ANOVA on $(\lambda_{\text{same}}, \beta)$ would be more informative but requires a full factorial design; due to compute limitations only an incomplete set of cells is available (Table 1), so we collapse all conditions into a single factor.

Cond.	Description	λ_{adj}	λ_{same}	β	Seeds
(a)	adj-only, plain BCE	1.0	0.0	1/52	0–4
(b)	same-only	0.0	1.0	1/52	0–4
(c)	multi-task, $\lambda_{\text{same}} = 1.0$	1.0	1.0	1/52	0–4
(d)	multi-task, $\lambda_{\text{same}} = 0.2$	1.0	0.2	1/52	0–4
(e)	adj-only, class-balanced BCE	1.0	0.0	1	0–4

Table 1: Unified ANOVA design. Each condition uses five fixed random seeds $\{0, 1, 2, 3, 4\}$. β controls the positive-class upweighting in the adjacency loss (Section 5.2): $\beta = 1/52$ is plain (unweighted) BCE, $\beta = 1$ is the textbook class-balanced weight ($w_+ = 52$). Condition (a) reuses existing runs `adj_only_seed_0-4`.

Table 2: Per-condition summary statistics for best-checkpoint ARI (5 seeds each). $\hat{\sigma} = 0.0154$; $\Delta_{\text{min}} \approx 0.0220$.

Condition	n	Mean ARI	SD	Min	Max
(a) adj-only, $\beta = 1/52$	5	0.5377	0.0165	0.5109	0.5553
(b) same-only	5	0.5409	0.0183	0.5139	0.5649
(c) multi-task, $\lambda_s = 1.0$	5	0.5574	0.0089	0.5506	0.5730
(d) multi-task, $\lambda_s = 0.2$	5	0.5315	0.0118	0.5154	0.5432
(e) adj-only, $\beta = 1$	5	0.4605	0.0190	0.4408	0.4831

9.3 Performance on the pretext task

Across all configurations tested, AUROC sits in the range 0.95–0.97 and AUPRC is on the order of 20× the random baseline of 0.019. Given a random adjacent pair and a random non-adjacent pair, the model ranks the adjacent one higher $\sim$ 95–97% of the time; the top-ranked pairs are almost all true adjacencies. By any reasonable standard the model has *nearly solved* the binary pretext task, and this holds regardless of the loss configuration. Downstream clustering ARI is meaningful but lower; representative failure cases are discussed in Section 10.

Figure 5 shows the ROC and precision-recall curves for the best checkpoint. The ROC corner sits well into the top-left, consistent with the high AUROC; the precision-recall curve drops more steeply, reflecting the low positive rate of $\sim$ 1.9%.

Figure 6 shows the score distribution broken down by true pair class. Three regimes are visible: cross-image pairs concentrate near zero, strictly adjacent pairs peak around 0.7–0.85, and same-image-but-not-adjacent pairs span the full $[0, 1]$ range. The model has implicitly learned a graded *same-source* signal rather than a strict binary adjacency predictor, which is precisely what the downstream spectral clustering step consumes.

9.4 Final test evaluation

The results above were obtained on validation batches used for checkpoint selection and are therefore subject to a mild selection bias. To obtain an unbiased estimate, we evaluated the best checkpoint from the ANOVA on 1000 independently sampled test batches. The test split contains 5,000 images; with batches of 10 images each image appears on average twice across the 1000 batches, so the evaluation is not fully independent at the image level, though each batch presents a distinct clustering problem. The chosen run was

Table 3: Welch one-way ANOVA omnibus test ($\alpha = 0.10$). * $p < \alpha$.

Source	F	df_1	df_2	p	sig
Between conditions	22.62	4.00	9.80	0.00	*

Table 4: Planned contrasts (Welch t-test, $\alpha = 0.10$). Estimate = mean(cond) − mean(a). d = Cohen's d. * $p < \alpha$.

Contrast	Estimate	95% CI	t	df	p	d	sig
(c) vs (a): same-image loss	0.02	[-0.0006, 0.0401]	2.36	6.20	0.06	1.49	*
(e) vs (a): loss tilt β	-0.08	[-0.1032, -0.0513]	-6.88	7.80	0.00	-4.35	*

`multitask_10_seed_2` ($\lambda_{\text{adj}} = 1.0$, $\lambda_{\text{same}} = 1.0$, $\beta = 1/52$, seed = 2), which achieved the highest validation ARI of 0.573.

Metric	**Mean**	**Std**
ARI	0.521	0.104
NMI	0.686	0.071
Purity	0.694	0.076
Adj. AUROC	0.948	—
Adj. F1	0.356	—

10 Failure Modes

Figure 7 shows four misclassified fragments from the best checkpoint. The model produces only an unordered partition into $k = 10$ clusters with arbitrary integer IDs; to relate them to source images for this visualisation we align cluster IDs to true source IDs by Hungarian matching on the validation batch (`scipy.optimize.linear_sum_assignment` on the 10×10 confusion matrix). The rightmost column then shows the source image whose fragments form the plurality of the cluster the misclassified fragment was assigned to. The matching is per-batch and uses the true labels; the model itself has no notion of source identity.

A reliable taxonomy of failure modes would require aggregating misclassified fragments across many batches, which we leave as follow-up work.

11 Limitations

- Adjacency is a local signal — it says nothing directly about fragments that are far apart within the same image. The model must generalise from local adjacency to global source identity.
- The graph has 12,720 edges. For larger grids or larger samples this grows quadratically and may become a bottleneck.
- The pretext metric and the task metric are misaligned. We train binary adjacency (only ~1.9% positives) but evaluate clustering. The same-image loss term partially

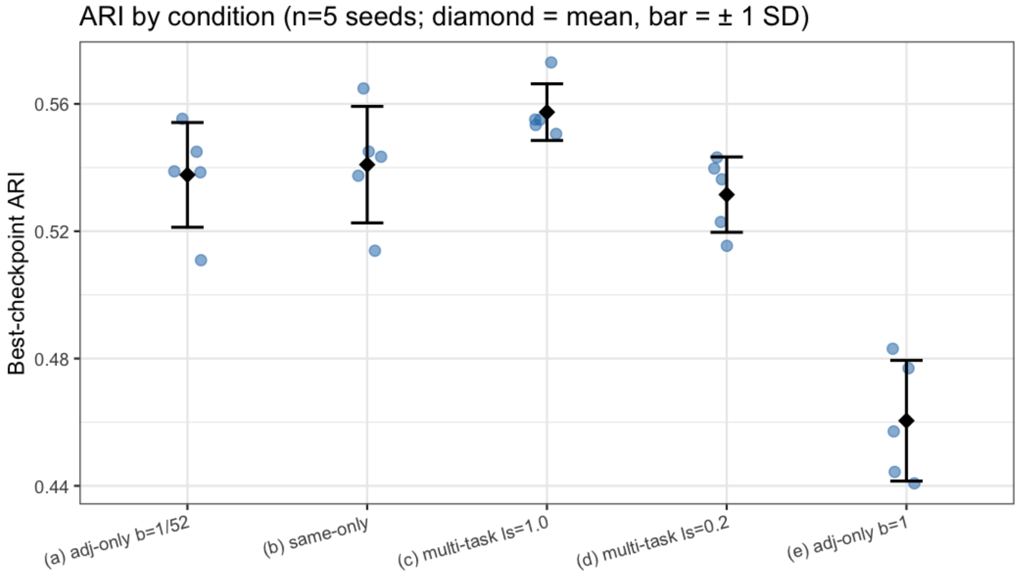


Figure 4: ARI per seed for each condition. Diamond = mean, bar = ±1 SD. Conditions (a)–(d) are tightly clustered; condition (e) ($\beta = 1$) sits ~ 0.08 ARI lower.

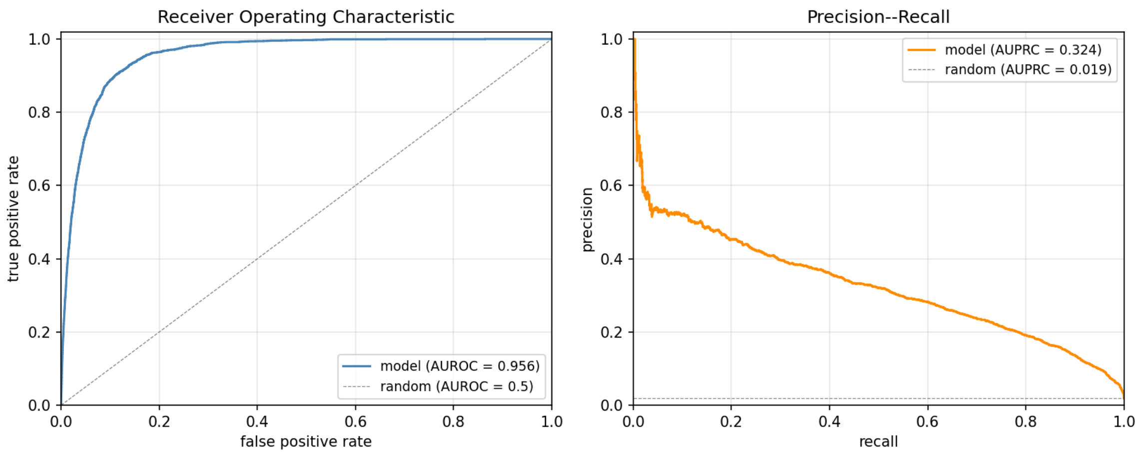


Figure 5: Threshold-free adjacency-prediction curves for the best checkpoint (`noise_seed_0`, $\beta = 0.01923$, $\lambda_{\text{adj}} = 1.0$, $\lambda_{\text{same}} = 1.0$). Curves aggregated over 10 fresh validation batches (127,200 pairs in total, with positive rate 0.0189). AUROC $= 0.956$ and AUPRC $= 0.324$; the AUPRC random baseline is the positive rate (≈ 0.019, dashed grey line).

closes this gap, but at the cost of about 0.01 AUROC and 0.10 AUPRC on the adjacency task itself. The model trades per-edge precision for a denser task-aligned signal.

12 Further Suggestions

Self-supervised pretrained encoder. DINOv2 (He et al. 2020) or another self-supervised image encoder could replace our trained-from-scratch CNN. Since these encoders use no labels, this stays within the constraints of the task. Whether the gain (richer features)

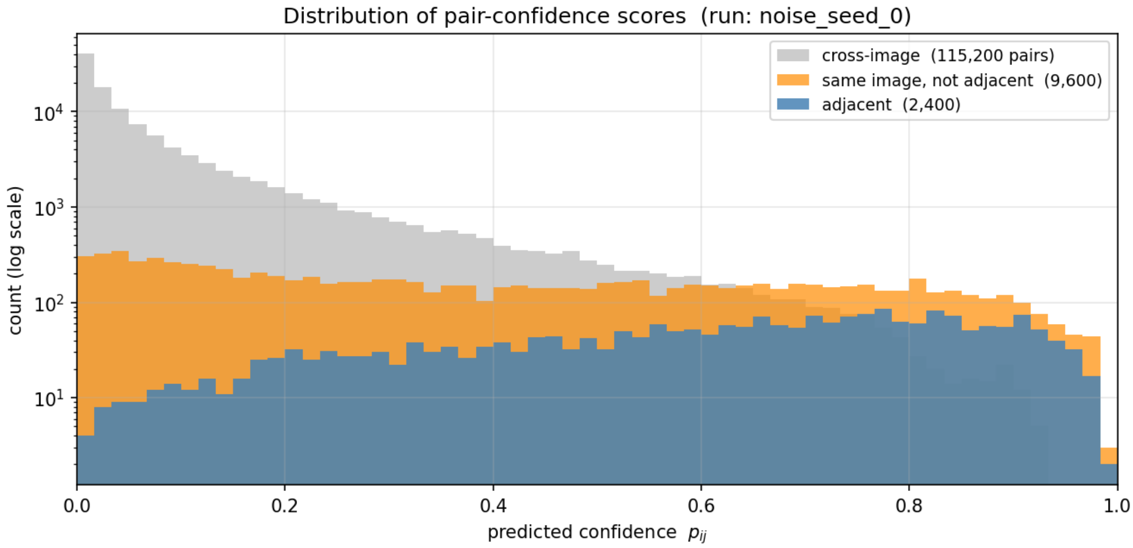


Figure 6: Distribution of the model's per-pair confidence scores p_{ij} on 10 fresh validation batches (`noise_seed_0`, $\beta = 0.0192$, $\lambda_{\text{adj}} = 1.0$, $\lambda_{\text{same}} = 1.0$). Pairs are split by their true class: adjacent (blue, 2,400 pairs), same source image but not adjacent (orange, 9,600), and cross-image (grey, 115,200). y-axis is logarithmic to make the rare classes visible.

outweighs the loss (features were trained at 224×224, not 16×16) is an open question.

Topology-aware clustering. The true partition is more constrained than "equal sizes": each cluster's induced subgraph is isomorphic to a $G \times G$ grid (16 nodes, 24 edges, a planar graph with maximum degree 4). A clustering algorithm that exploited this structure, e.g. a small GNN trained to find equipartitions whose induced subgraphs are grid-like, could disambiguate borderline assignments that pure spectral clustering misses. Implementation cost is significant; the marginal benefit beyond what the model's already-strong soft signal recovers is unclear.

Color-histogram baseline. A non-learned baseline using per-fragment color histograms with chi-square or EMD distance, followed by spectral clustering, would anchor the learned model's ARI against a meaningful reference. We expect this to be competitive on uniform-region fragments and weak on textured ones; a controlled comparison would quantify how much of the 0.57 ARI comes from learned representation vs. trivial color matching.

References

Bromley, Jane et al. (1994). "Signature Verification Using a Siamese Time Delay Neural Network". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 6, pp. 737–744.

Caron, Mathilde, Piotr Bojanowski, et al. (2018). "Deep Clustering for Unsupervised Learning of Visual Features". In: *European Conference on Computer Vision (ECCV)*, pp. 132–149.

Caron, Mathilde, Ishan Misra, et al. (2020). "Unsupervised Learning of Visual Features by Contrasting Cluster Assignments". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33, pp. 9912–9924.

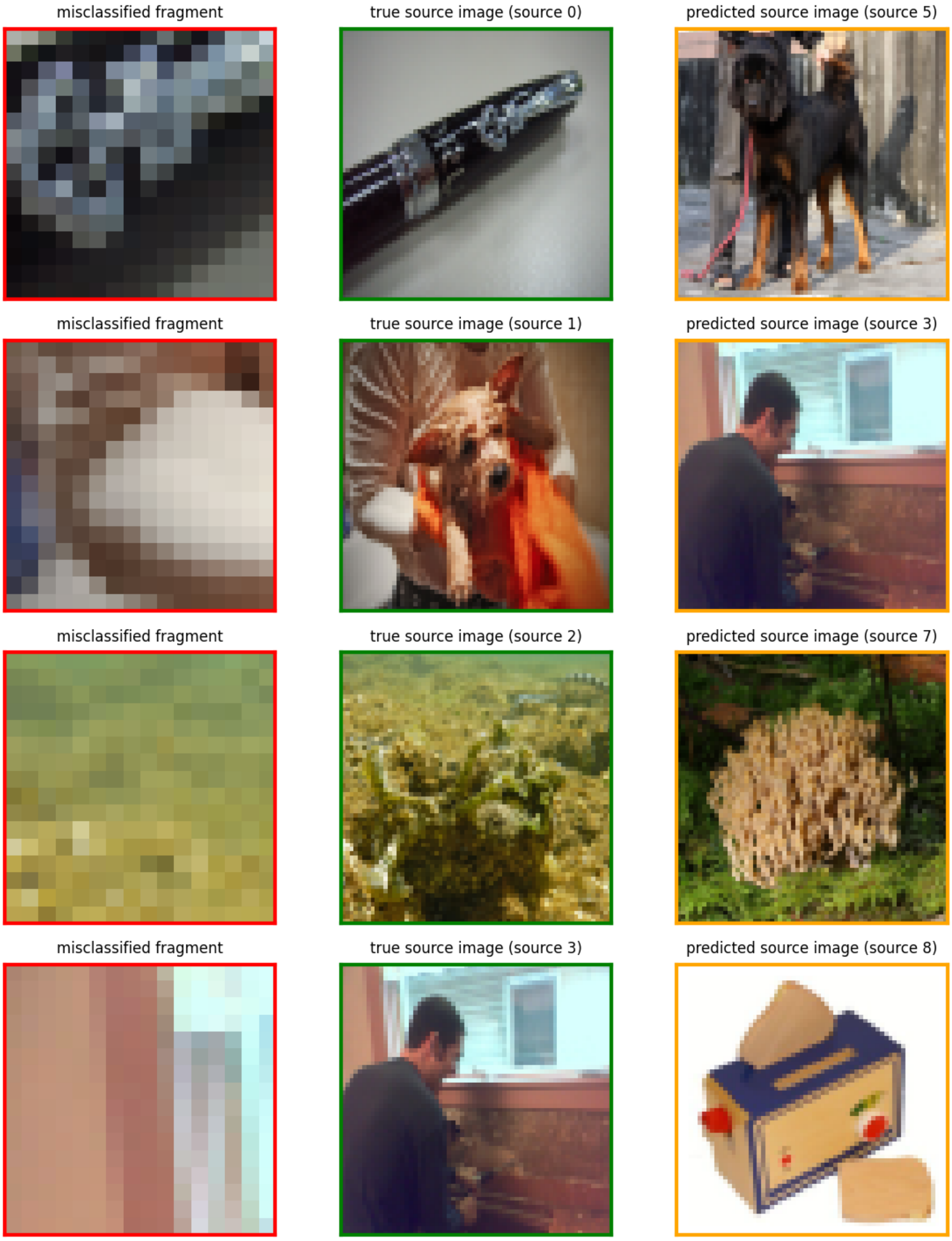


Figure 7: Selected misclassified fragments from the best checkpoint (`multitask_10`, best ARI $= 0.570$, $\beta = 0.01923$, $\lambda_{\text{adj}} = 1.0$, $\lambda_{\text{same}} = 1.0$). On the validation batch shown, 51 of 160 fragments were assigned to the wrong cluster after Hungarian matching of predicted to true labels. Each row shows: the misclassified fragment (left, red), the true source image (middle, green), and the source image the model assigned it to (right, orange).

Chen, Ting et al. (2020). "A Simple Framework for Contrastive Learning of Visual Representations". In: *International Conference on Machine Learning (ICML)*, pp. 1597–1607.

Chrabaszcz, Patryk, Ilya Loshchilov, and Frank Hutter (2017). "A Downsampled Variant of ImageNet as an Alternative to the CIFAR Datasets". In: *arXiv preprint arXiv:1707.08819*.

Conneau, Alexis et al. (2017). “Supervised Learning of Universal Sentence Representations from Natural Language Inference Data”. In: *Empirical Methods in Natural Language Processing (EMNLP)*, pp. 670–680.

He, Kaiming et al. (2020). “Momentum Contrast for Unsupervised Visual Representation Learning”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9729–9738.

Noroozi, Mehdi and Paolo Favaro (2016). “Unsupervised Learning of Visual Representations by Solving Jigsaw Puzzles”. In: *European Conference on Computer Vision (ECCV)*. Springer, pp. 69–84.